

我完成了所有内容.
还有音频播放, echo, etc.
信号机制锐意制作中

前言

豪德, 我现在知道 timer 的坑在哪里了.

必答题

Context

- 在哪里 & 从哪来

即为当前栈指针, `__am_asm_trap` 开始的时候在当前栈上开了一个 context 的空间然后保存 gpr | mepc | mstatus | mcause 到栈上, 按照 context 的结构构造出一个 context 结构体, 调用 `__am_irq_handle` 的时候把 a0 设置为 sp 以传递 context*

- 在哪赋值

```
MAP(REGS, PUSH)
```

```
csrr t0, mcause
csrr t1, mstatus
csrr t2, mepc
```

```
STORE t0, OFFSET_CAUSE(sp)
STORE t1, OFFSET_STATUS(sp)
STORE t2, OFFSET_EPC(sp)
```

- 联系

- riscv.h 定义了一个 context 结构体的规范, trap.S 和 c 同时遵守以实现数据沟通
- 上述讲义: 告诉我要干啥
- 新指令: 用来访问 csr, 使 csr 的值能够被保存到 context 中

Yield

- `yield()` : 把异常号 (-1) 放进 a7, 然后发动 `ecall`
- `ecall`: 发动 `isa_raise_intr`, 传递异常号
- `isa_raise_intr()`: 保存 mepc, 设置 mcause, 跳转到 mtvec (`__am_asm_trap`)
- `__am_asm_trap` : 保存 context, 调用 `__am_irq_handle()`
- `__am_irq_handle()` : 如果存在 user_handle, 则根据 mcause 进行事件分发, 然后把事件和 context 传给 user_handle
- user_handle : 发现是 EVENT_YIELD, 打个 y, 并返回原先 context
- `__am_irq_handle()` : 返回 user_handle 返回的 context
- `__am_asm_trap` : 从 context 中恢复 gpr, mepc, mstatus, mcause, 发动 `mret`
- `mret`: 跳转到 mepc 的下一条指令, 返回原先执行流

- # 注: 我认为 mepc 应当是 pc after exception, 而非 pc when exception. pa3 中单纯的 ecall 回来后下一条指令一定是 pc+4, 但是 pa4 中加入硬件中断后则不一定. eg. 在 jmp | mret | .etc 后产生硬件中断, 保存 pc when exception 则丢弃了本来的下一条指令的信息. 但是保存 pc after exception 则可以保证下一条指令的信息不会丢失.

Hello

- 程序加载相关
 - 一开始在文件(ramdisk)里, 通过 loader 读到内存中. elf 文件的头部有 elf 文件的组织结构, 指示了程序的不同 segment 应当加载到内存的哪个地址上, 第一条指令是 Elf_Ehdr 中的 e_entry 所指示的地址, 在 c 代码中通过函数调用以执行该第一条指令.
- 字符输出相关
 - printf(格式字符串, 要格式化的值) → write(stdout, 格式化完的字符串) → _write(stdout, 格式化完的字符串) → _syscall(系统调用号:SYS_write, 其他参数):
把调用号和参数放进寄存器, 执行 ecall → 中断处理流程... → do_event(事件,上下文):
发现是事件是 syscall → do_syscall(上下文): 从上下文中捞出调用号和参数 → sys_write(stdout, 字符串) → fs_write(stdout, 字符串) → serial_write(字符串):
逐字符输出 → putchar(字符): 向串口 mmio 区域写入字符 → nemu 执行写入发现是 mmio 区域, 写入对应位置后调用 callback, 输出字符到终端

槽点

最近席卷计算机领域的, 就要数 2018 年 1 月爆出的 Meltdown 和 Spectre 这两个大名鼎鼎的硬件漏洞了.

哥们你这有点古老啊, 不整整最近 Linux 的打印机 RCE 吗?

思考题

CTE 还可能切换进程, 返回别人的 context, 所以要保存全部 reg
CISC 简化了软件设计, RISC 简化了硬件设计, 给了软件自由度. 合不合理? 看当时环境!
堆和栈是操作系统分配的, 放到文件内会固化可使用的内存空间且使文件巨大
头上的 magic number 不一样
有的数据不需要包含在文件内, 但是载入内存的时候要分配空间(eg. .bss), 清零是为了避免读到别人用的脏数据
会产生资源竞争和状态不一致(eg. malloc 维护的内存分配要 OS 统一管理)
a0 是函数的参数传递, 用 a7 是为了避免和既有约定冲突(?
/dev/events 不可定位, 所以用 open
float to fixedpt: 把对应 bits 捞出来, 按 IEEE754 标准转换成 fixedpt
LD_PRELOAD: 优先加载动态链接库

秘技: 笑死, uint aroundwarp 了

唉唉, 代码混淆. 最简单方案: 关键字算 hash 然后替换